

Aurora Conformance Loop Speaking Notes

Two-slide talk track for conformance iteration, compute economics, and model-assisted assessment.

Slide 1: Conformance Improvement Loop

The core idea is that we are treating conformance as an evidence-driven engineering loop, not as subjective arcade imitation.

We start by ingesting reference material: video, audio, frames, sprites, timing traces, and stage behavior. That gives us the target.

Then we convert the reference into measurements: waveform comparisons, spectral analysis, motion paths, pressure windows, contact sheets, and timing traces.

The model's role is not just to suggest random gameplay changes. Its highest value is helping us design better metrics, diagnose gaps, propose scorer logic, and choose the next-best experiment.

Once we have a measurable hypothesis, we spend local compute: CPU and browser harnesses for deterministic gameplay testing, and GPU or visual compute for frame and image comparison.

Then we make a candidate improvement. That could be gameplay tuning, audio feedback, graphics work, or an improvement to the ingestion harness itself.

After that, we score it. We check conformance movement, release gates, and no-regression guardrails.

Finally, we document economics: wall time, CPU/GPU/API usage, artifact growth, and return on effort. That feeds the next cycle, so we keep improving the system's judgment over time.

Key line: We are not trying to spend more compute. We are trying to get more arcade fidelity, better game feel, and more reusable harness logic per unit of compute.

Slide 2: How We Optimize Conformance ROI

This second slide breaks the loop into workstreams.

Reference ingestion is foundational because it defines what we are trying to match. Without that, we are tuning from memory.

Audio analysis is currently one of the highest-value areas because Aurora's audio identity and event feedback are still weaker than core movement or combat.

Gameplay analysis is where CPU and browser harnesses matter most. This is how we test pressure, pacing, level arc, stage shape, and repeatability.

Visual analysis is where GPU or image-style computation becomes more useful. We want sprite alignment, contact sheets, alien entry, UI surfaces, and overall arcade feel to become measurable rather than purely subjective.

Model-assisted planning helps us decide what to measure next and how. We want the model improving the evaluation system, not just giving us one-off design opinions.

Candidate implementation is where Codex and the codebase turn the evidence into actual changes.

Regression verification keeps us from damaging areas that are already strong, like movement, combat responsiveness, and capture/rescue rules.

Economics tracking is what makes this scalable. We want to know whether three hours of CPU, GPU, or model/API work created a measurable score gain, better harness logic, or just noise.

Decision rule: Pick the work with the best expected conformance gain per unit of effort, while preserving guardrails.

Current highest-value sequence: audio identity and event feedback, then level arc and encounter shape, then visual look-and-feel scoring, then Stage pressure replay precision.